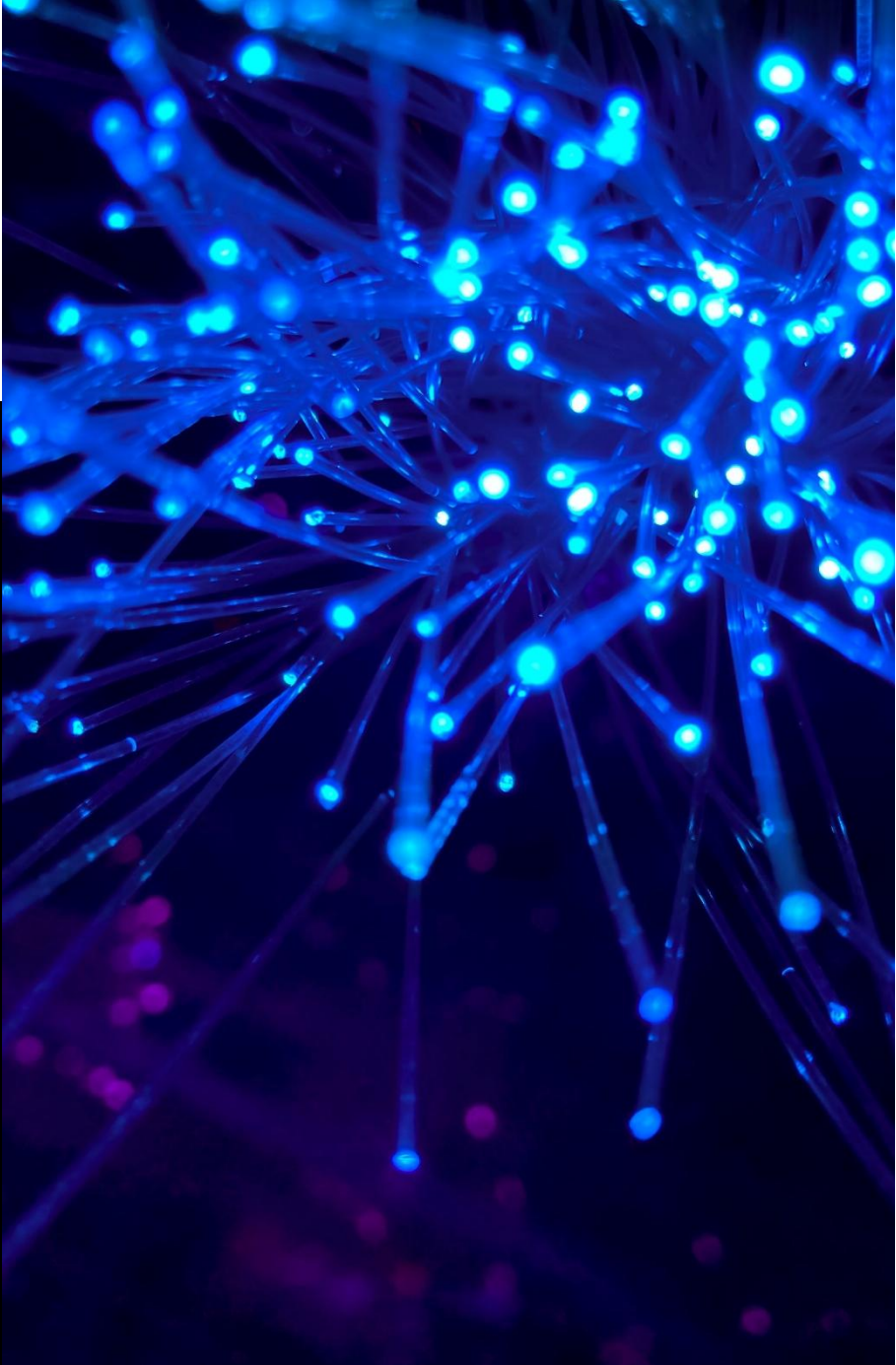




Minería de datos: textos

Minería de datos de texto o Minería de textos (Text Mining)

Es el proceso de transformar texto no estructurado en un formato estructurado para identificar patrones significativos y extraer información.

Área multidisciplinar:

- Estadística
- Lingüística computacional: formalismos del funcionamiento del lenguaje natural.
- Aprendizaje automático

Minería de textos - Proceso

1. Recolección de datos
2. Preprocesamiento
3. Enriquecimiento
4. Transformación
5. Extracción de características

Datos

Limpiar & Estructurar
- palabras “sin contenido”
- signos de puntuación
- lematización

Añadir tags

Transformar mediante
algoritmos -> convertir en
vectores numéricos

i

Minería de textos - Aplicaciones

- Extracción de información
- Visualización: resaltar la información
- Clasificación de textos: categorías
- Análisis de sentimientos

Minería de textos

Vamos a usar una DB con opiniones generadas en MongoDB.

Crear un base de datos "minería" con dos colecciones:

- "opinionesmvit"
 - "opinionespuls"
- e importar los archivos .json correspondientes.

NLTK

Documentation

Search

Natural Language Toolkit

NLTK Documentation
API Reference
Example Usage
Module Index

NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to **over 50 corpora and lexical resources** such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active **discussion forum**.

<https://www.nltk.org/>

nltk 3.9.4

`pip install nltk`

✓ Versión más reciente

Publicación: 24 mar 2026

Natural Language Toolkit

Navegación

Descripción de proyecto

Histórico de versiones

Descripción de proyecto

The Natural Language Toolkit (NLTK) is a Python package for natural language processing. NLTK requires Python 3.10, 3.11, 3.12, 3.13, or 3.14.

<https://pypi.org/project/nltk/>

1. Preparación de datos: traer el campo comentario de BBDD

Vamos a usar una DB con opiniones generadas en MongoDB.

Crear un base de datos "minería" con dos colecciones:

- "opinionesmvit"
- "opinionespuls"

e importar los archivos .json correspondientes.

```
import pymongo
import sys
import os
import dotenv
import pandas as pd
import nltk
```

Conectar a MongoDB y traer los datos indicados a continuación. Almacenar en un DataFrame.

```
resultsmv = list(col_opismv.find({}, {"_id":0, "id":1,
"comentario":1, "valoracion":1, "recompraria":1}))
print(resultsmv)
resultspu = list(col_opispu.find({}, {"_id":0, "id":1,
"comentario":1, "valoracion":1, "recompraria":1}))

#Convertir a DF - opiniones texto no estructurado
df_datamv = pd.DataFrame(resultsmv)
df_datapu = pd.DataFrame(resultspu)
```

2. Preprocesamiento de texto (Limpieza)

- Convertir a minúsculas
- Quitar puntuación
- Tokenizar: separar palabras de signos de puntuación (a diferencia de un split, entiende las reglas del lenguaje)
 - dependiendo de la versión de python y para idiomas distintos de inglés puede necesitar descargar punkt o punkt_tab
- Eliminar stopwords (palabras comunes que no aportan significado como "que", "y", "la")

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from collections import Counter
import string

stop_words = set(stopwords.words('spanish'))

def clean_text(text, testing=None):
    # Convertir a minúsculas
    text = text.lower()

    # Eliminar puntuación
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Tokenización
    tokens = word_tokenize(text, language="spanish")

    # Eliminar stopwords
    clean_tokens = [word for word in tokens if word not in stop_words]
    return " ".join(clean_tokens)

df_datamv["comentlimp"] = df_datamv["comentario"].apply(clean_text)
df_datapu["comentlimp"] = df_datapu["comentario"].apply(clean_text)
```

3. Análisis de frecuencia:

Análisis de las palabras más frecuentes

```
all_wordsmv = " ".join(df_datamv["comentlimp"]).split()
word_freqmv = Counter(all_wordsmv)
top_wordsmv = word_freqmv.most_common(10)
print(f"Las 10 palabras más comunes: {top_wordsmv}")

all_wordspu = " ".join(df_datapu["comentlimp"]).split()
word_freqpu = Counter(all_wordspu)
top_wordspu = word_freqpu.most_common(10)
print(f"Las 10 palabras más comunes: {top_wordspu}")
```


Opcional: guardar tokens en MongoDB

```
# Opcional _ crear un dataframes con los tokens en formato array y guardar en MongoDB
# Ejemplo: pulsómetro

df_datapu_watok = df_datapu
df_datapu_watok["arrtokenspu"] = df_datapu_watok["comentlimp"].str.split(" ")

# Ejecutar múltiples operaciones de escritura con bulk_write
wops = []

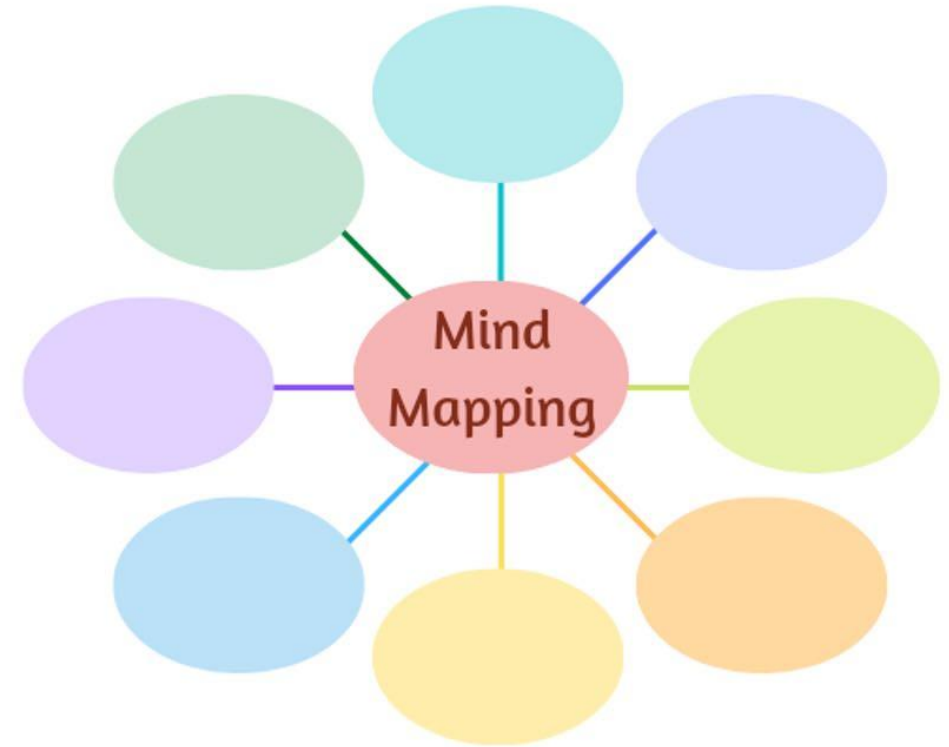
for _, row in df_datapu_watok.iterrows():
    wops.append(
        pymongo.UpdateOne(
            {"id": row["id"]},
            {"$set": {"tokens": row["arrtokenspu"]}}
        )
    )

col_opispu.bulk_write(wops)
```

4. Visualizar resultados



WordCloud



MindMap

4.1 WordCloud:

pip install wordcloud

- generate(): Es el motor principal que procesa el texto y asigna tamaños según la frecuencia de cada palabra
- stopwords: limpia palabras, especialmente importante si la limpieza previa es deficiente o inexistente
- interpolation='bilinear': Hace que las palabras se vean más nítidas y suaves en la imagen final



<https://pypi.org/project/wordcloud/>

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud

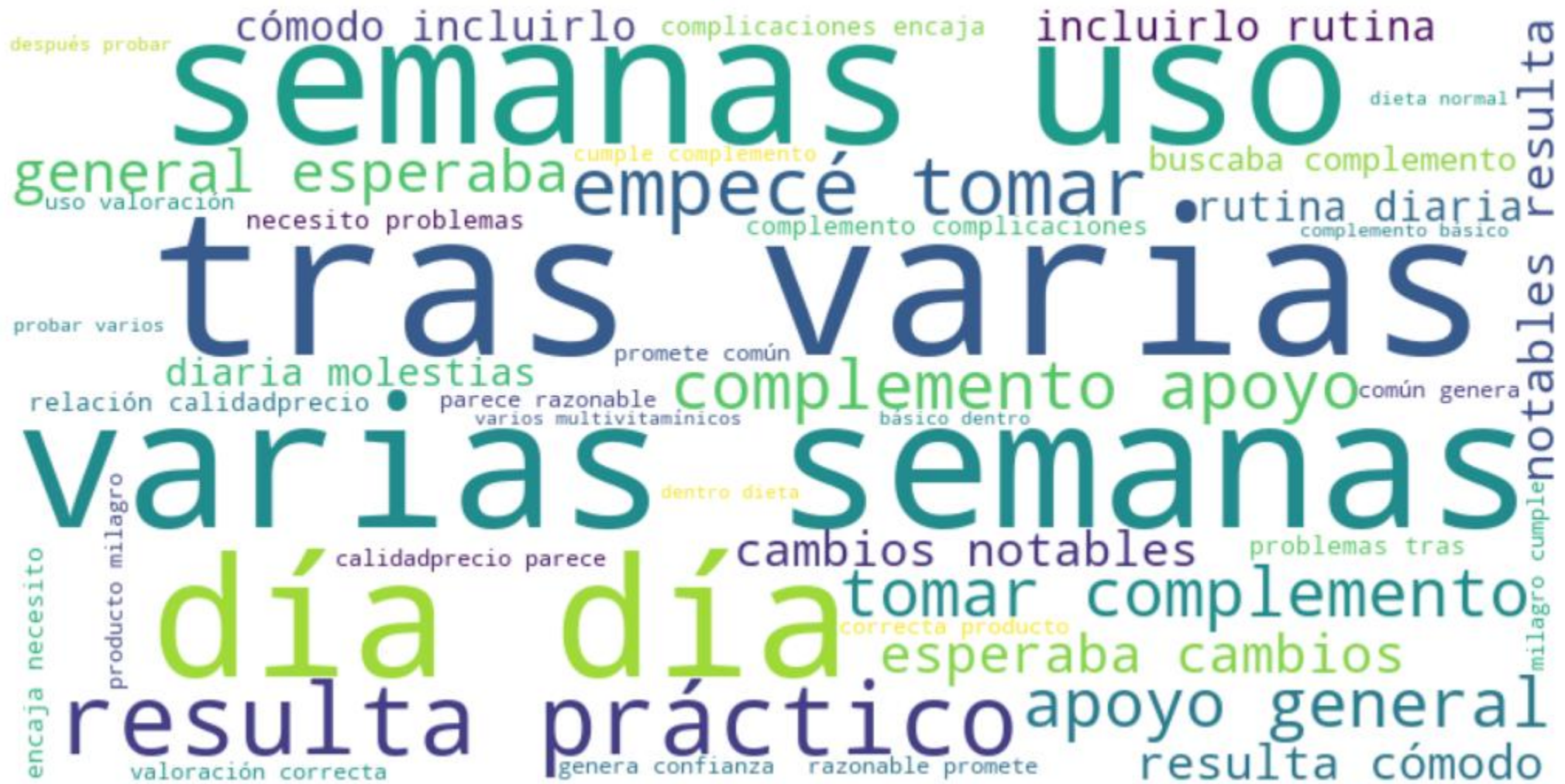
stopwords_add = set([])
cleandata_joinmv = " ".join(df_datamv["comentlimp"])

wcloudmv = WordCloud(
    width=800,
    height=400,
    background_color='white',
    stopwords=stopwords_add,
    colormap='viridis', #coolwarm
    min_font_size=10
).generate(cleandata_joinmv)

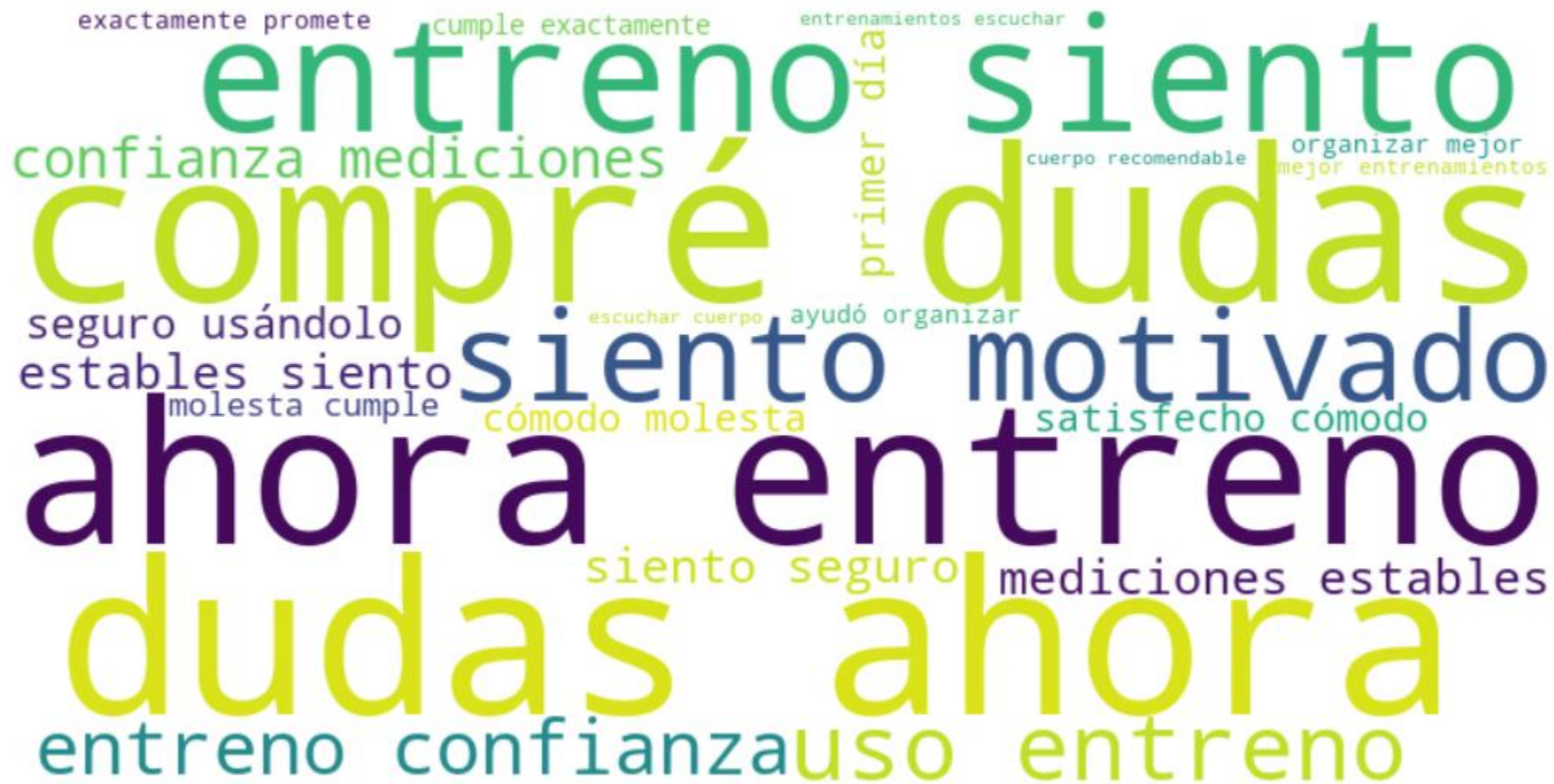
#Visualización con Matplotlib
plt.figure(figsize=(10, 5), facecolor=None)
plt.imshow(wcloudmv, interpolation='bilinear')
plt.axis("off")
plt.tight_layout(pad=0)

plt.show()
```

Hacer el proceso para ambos casos



Ejecutar varias veces



MongoDB Atlas

- > Visualization
- > Dashboards
- > Add chart
 - Chart type: WordCloud
 - Field: tokens
 - Method: unwind array
 - Admite queries para filtrar

Comentarios

token frequency



Para 100 docs

4.1 WordCloud:

Vamos a hacer que dependa de las frecuencias con las que aparecen las distintas palabras.

```
wcloudmvf = WordCloud(  
    width=800,  
    height=400,  
    background_color='white',  
    stopwords=stopwords_add,  
    colormap='viridis', #coolwarm  
    min_font_size=10,  
    relative_scaling=1  
)  
.generate_from_frequencies(word_freqmv)
```

Hacer el proceso para ambos casos



Análisis de minería en función de la valoración dada en las opiniones:

Opiniones: 4-5 Estrellas



Opiniones: 1-2 Estrellas



A word cloud visualization of user feedback. The words are arranged in a circular pattern, with the most prominent words in the center and smaller words towards the edges. The colors of the words vary, including shades of blue, green, yellow, and purple. The words represent various aspects of user experience, such as product quality, usability, and customer service.

Key words and phrases visible in the word cloud include:

- bien** (large, dark blue)
- uso** (large, dark blue)
- entrenamiento** (large, green)
- esperaba** (large, teal)
- aunque** (large, dark blue)
- si** (large, dark blue)
- entreno** (large, green)
- mediciones** (medium, teal)
- pulsómetro** (medium, green)
- ahora** (medium, yellow)
- compré** (medium, green)
- ayudó** (medium, green)
- desconexión** (medium, green)
- confianza** (medium, purple)
- totalmente** (medium, green)
- correctamente** (medium, teal)
- usándolo** (medium, teal)
- escuchar** (medium, teal)
- entrenamientos** (medium, yellow)
- puntual** (medium, teal)
- satisfecho** (medium, teal)
- vender** (medium, teal)
- mejor** (medium, yellow)
- exactamente** (medium, green)
- correcto** (medium, teal)
- cómodo** (medium, green)
- organizar** (medium, green)
- devuelto** (medium, green)
- nota** (medium, green)
- reales** (medium, green)
- numeros** (medium, green)
- dudas** (medium, green)
- estables** (medium, teal)
- llamar** (medium, teal)
- excelente** (medium, teal)
- recomendable** (medium, teal)
- semanas** (medium, green)
- motivado** (medium, purple)
- ver** (medium, green)
- entrenos** (medium, green)
- complicaciones** (medium, green)
- hace** (medium, teal)
- real** (medium, teal)
- seguro** (medium, teal)
- hacer** (medium, teal)
- solo** (medium, teal)
- alguna** (medium, purple)
- despreocupó** (medium, green)
- primer** (medium, green)
- batería** (medium, teal)
- decepcionado** (medium, teal)
- buscando** (medium, teal)
- sorprendido** (medium, teal)
- eficaz** (medium, green)
- mal** (medium, teal)
- varias** (medium, teal)
- motivó** (medium, green)
- clave** (medium, yellow)
- justo** (medium, green)
- tiempo** (medium, teal)
- pulsaciones** (medium, teal)
- precio** (medium, teal)
- atención** (medium, teal)
- innecesarias** (medium, teal)
- cuerpo** (medium, teal)
- trabajo** (medium, teal)
- después** (medium, teal)
- creo** (medium, teal)
- ganas** (medium, teal)
- promete** (medium, green)
- entrenar** (medium, teal)
- producto** (medium, teal)
- fiabiles** (medium, teal)
- sencillo** (medium, teal)

Análisis de minería en función de la valoración dada en las opiniones:

Opiniones: 4-5 Estrellas



Opiniones: 1-2 Estrellas



4.2 MindMap:

pip install networkx

El mapa mental es una representación tipo grafo, de modo que definiremos los elementos para mostrar esa idea de grafo.

Vamos a definir un grafo con un nodo central, que en nuestro ejemplo va a ser el producto que genera las opiniones.



<https://pypi.org/project/networkx/>

```
import networkx as nx
import matplotlib.pyplot as plt
```

```
# Crear el grafo para el mapa mental
graph = nx.Graph()
```

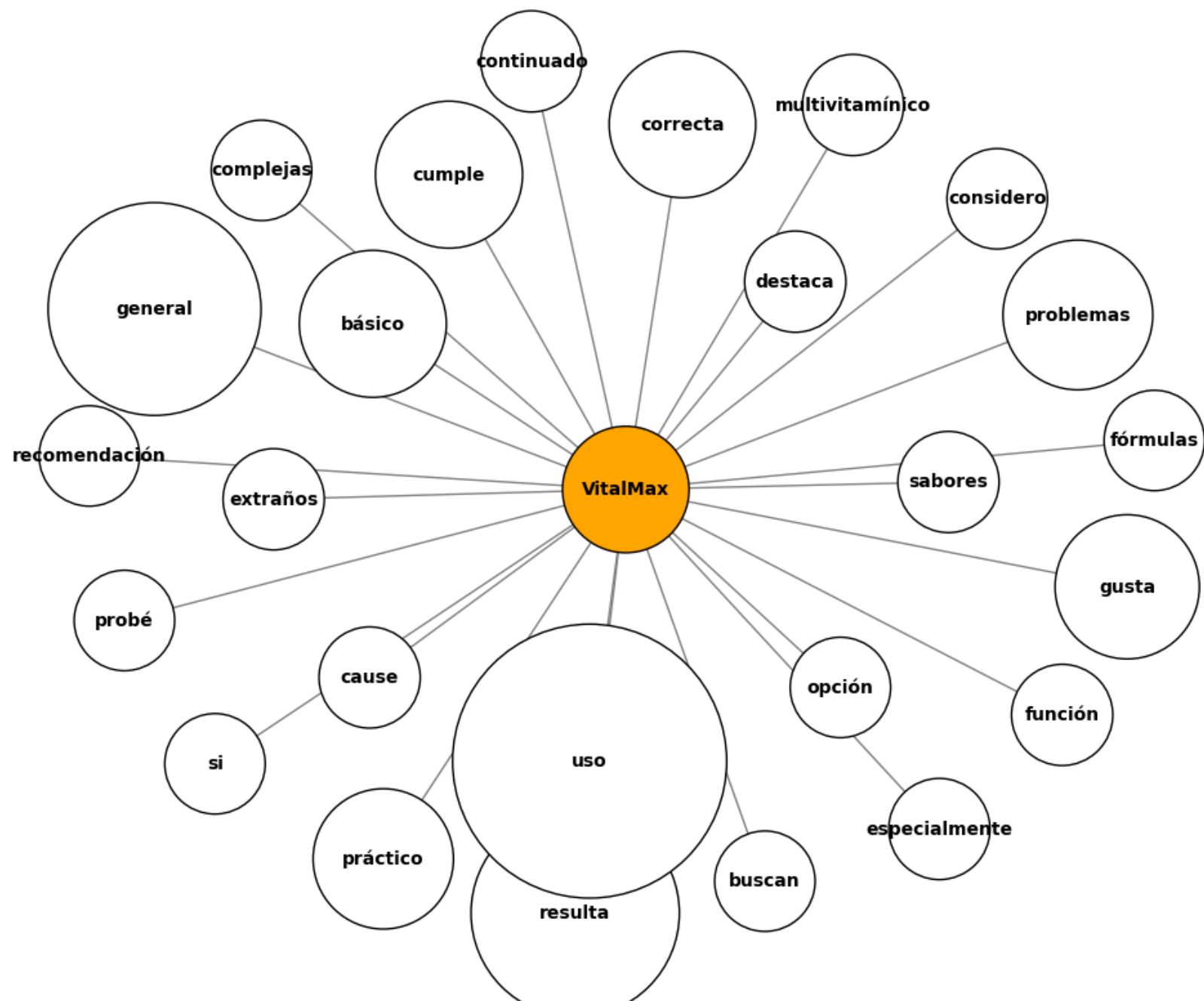
```
# Definir el nodo central
central_node = "VitalMax"
```

```
# Definir listas para almacenar los tamaños y colores de cada nodo
nodes = [central_node]
sizes = [5000] # tamaño fijo para nodo central
colors = ['orange']
```

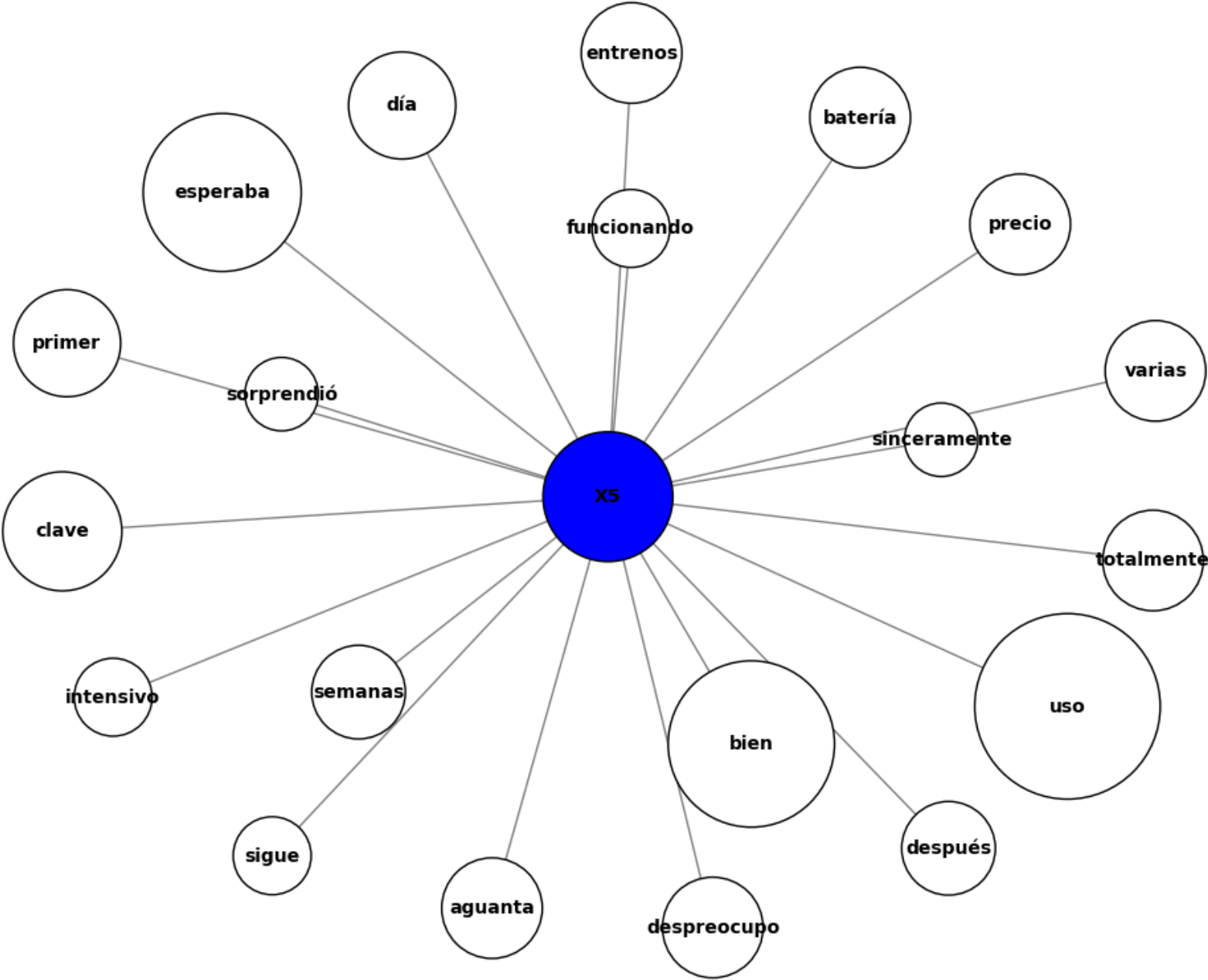
```
for word, count in list(word_freqmv.items())[:25]:
    #edge añade las conexiones entre nodos
    graph.add_edge(central_node, word)
    nodes.append(word)
    # El tamaño depende de la frecuencia
    sizes.append(count * 50)
    colors.append('white')
```

```
# Visualización
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(graph, k=0.5)
nx.draw(graph, pos,
        labels={n: n for n in nodes},
        nodelist=nodes,
        node_size=sizes,
        node_color=colors,
        font_size=10,
        font_weight='bold',
        edge_color='grey',
        edgecolors = "black")
```

```
plt.title("Mapa Mental")
plt.show()
```



Mapa Mental



Minería de textos – Análisis de sentimientos

Minería de textos – Análisis de sentimientos

Para el análisis de sentimientos las librerías se basan en diferentes modelos de lenguaje.

NLTK.sentiment & VADER

PySentimiento & RoBERTuito

Emociones semánticas

Minería de textos – Análisis de sentimientos

NLTK.sentiment & VADER

VADER es un módulo dentro de NLTK específicamente diseñado para sentimientos en redes sociales (textos cortos, informales, emojis).

- Se basado en **léxico y reglas**: utiliza una lista de palabras clasificadas como positivas o negativas, además de reglas para negaciones ("no es bueno") o intensificadores ("muy bueno").
- Es rápido, no requiere entrenamiento y funciona muy bien "out-of-the-box" para **inglés**.

VADER (Valence Aware Dictionary and sEntiment Reasoner) es mejor para detectar el sentimiento en textos cortos como tuits, reseñas de productos o comentarios de usuarios que contienen jerga, emojis y abreviaturas.

Las puntuaciones individuales de palabras se combinan para calcular **la puntuación compuesta**, que es un valor normalizado entre -1 y +1 que representa el sentimiento general:

- **> 0,05_0,35**: Sentimiento positivo
- **< -0,05_-0,35**: Sentimiento negativo
- **entre las anteriores**: Sentimiento neutral

```

nltk.download("vader_lexicon")
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

#los rangos de asignación más o menos restrictivos
def analizar_emocion(texto):
    scores = sia.polarity_scores(texto)
    if scores["compound"] >= 0.35:
        return "POS"
    elif scores["compound"] <= -0.35:
        return "NEG"
    else:
        return "NEU"

```

```

df_datamv["sentimiento_nltk"] =
df_datamv["comentario"].apply(analizar_emocion)
df_datapu["sentimiento_nltk"] =
df_datapu["comentario"].apply(analizar_emocion)

#Contabilizar por tipo
print(df_datamv["sentimiento_nltk"].value_counts())
print(df_datapu["sentimiento_nltk"].value_counts())

# Ver consistencia: Promedio de puntuaciones por sentimiento
print(df_datamv.groupby("sentimiento_nltk")['valoracion'].mean())
print(df_datapu.groupby("sentimiento_nltk")['valoracion'].mean())

```

sentimiento_nltk	
NEU	773
NEG	227

sentimiento_nltk	
NEG	3.383260
NEU	3.332471

Multivit.

sentimiento_nltk	
NEU	248
NEG	52

sentimiento_nltk	
NEG	3.961538
NEU	3.302419

Pulsom.

Minería de textos – Análisis de sentimientos

PySentimiento & RoBERTuito

BERT

BERT (Bidirectional Encoder Representations from Transformers) es un modelo de lenguaje desarrollado por Google en 2018 que cambió el procesamiento del lenguaje natural (PLN) al entender el contexto de las palabras en ambas direcciones.

RoBERTa

RoBERTa (Robustly optimized BERT approach) es una variante optimizada de **BERT**, entrenada con más datos y durante más tiempo por Facebook (Meta), lo que suele mejorar su rendimiento.

RoBERTuito

RoBERTuito es un modelo de lenguaje preentrenado diseñado específicamente para contenido generado por usuarios en español, entrenado siguiendo las directrices de **RoBERTa** con 500 millones de tweets.

Minería de textos – Análisis de sentimientos

PySentimiento & RoBERTuito

La librería PySentimiento utiliza principalmente **RoBERTuito** como modelo base para el análisis de sentimientos y emociones en español.

Optimizado para redes sociales, manejando bien el lenguaje coloquial, emojis y abreviaciones.

Asignación (pref. > 0.5):

- **Positivo:** Prob. POS > (NEG y NEU).
- **Negativo:** Prob. NEG > (POS y NEU).
- **Neutro:** Prob. NEU > (POS y NEG).

```
SentimentOutput(output=POS, prob={NEG: 0.1, NEU: 0.2, POS: 0.7})
```

pip install pysentimiento

pysentimiento: A Python toolkit for Sentiment Analysis and Social NLP tasks

[run tests](#) [no status](#) [Open in Colab](#)

A Transformer-based library for SocialNLP tasks.

Currently supports:

Task	Languages
Sentiment Analysis	es, en, it, pt
Hate Speech Detection	es, en, it, pt
Irony Detection	es, en, it, pt
Emotion Analysis	es, en, it, pt
NER & POS tagging	es, en
Contextualized Hate Speech Detection	es
Targeted Sentiment Analysis	es

Just do `pip install pysentimiento` and start using it:

<https://github.com/pysentimiento/pysentimiento>

```
from pysentimiento import create_analyzer
```

```
# Inicializar el analizador de sentimientos en español  
analyzer = create_analyzer(task="sentiment", lang="es")
```

```
# Aplicar al DF (puede tardar un poco en analizar todas las opiniones)  
df_datamv["sentimiento_IA"] = df_datamv["comentario"].apply(lambda x: analyzer.predict(x).output)  
df_datapu["sentimiento_IA"] = df_datapu["comentario"].apply(lambda x: analyzer.predict(x).output)
```

```
#Contabilizar por tipo  
print(df_datamv["sentimiento_IA"].value_counts())  
print(df_datapu["sentimiento_IA"].value_counts())  
  
# Ver consistencia: Promedio de puntuaciones por sentimiento  
print(df_datamv.groupby("sentimiento_IA")['valoracion'].mean())  
print(df_datapu.groupby("sentimiento_IA")['valoracion'].mean())
```

Contabilizados

MULTIVIT.

```
sentimiento_ntlk  
NEU    773  
NEG    227
```

```
sentimiento_IA  
POS    883  
NEU    117
```

PULSOM.

```
sentimiento_ntlk  
NEU    248  
NEG     52
```

```
sentimiento_IA  
POS    173  
NEG     65  
NEU     62
```

Consistencia: promedio puntuación

```
sentimiento_ntlk  
NEG    3.383260  
NEU    3.332471
```

NTLK

```
sentimiento_IA  
NEU    3.290598  
POS    3.351076
```

PySentimiento

```
sentimiento_ntlk  
NEG    3.961538  
NEU    3.302419
```

NTLK

```
sentimiento_IA  
NEG    1.953846  
NEU    2.903226  
POS    4.150289
```

PySentimiento

Para extraer las probabilidades numéricas de pysentimiento, debemos acceder al atributo .probas del objeto que devuelve el analizador. Devolverá un diccionario de probabilidades.

```
{'POS': 0.98, 'NEU': 0.01, 'NEG': 0.01}
```

Permite ver como de "seguro" está el modelo de su predicción.

¿Para qué sirve esto? **Detección de Sarcasmo:**

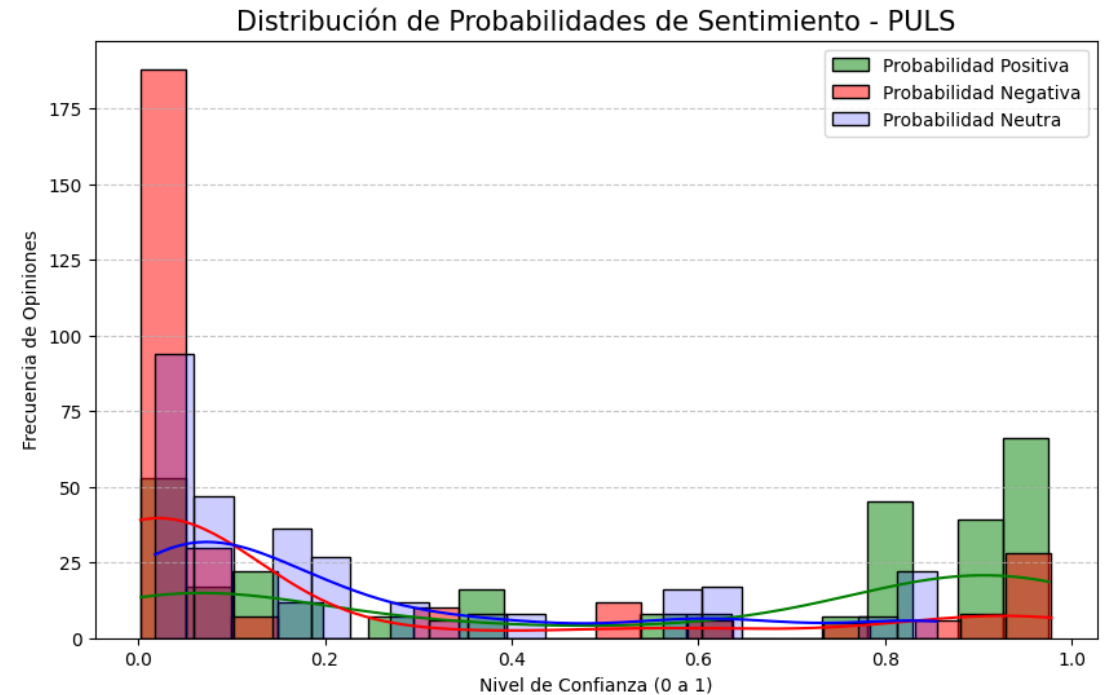
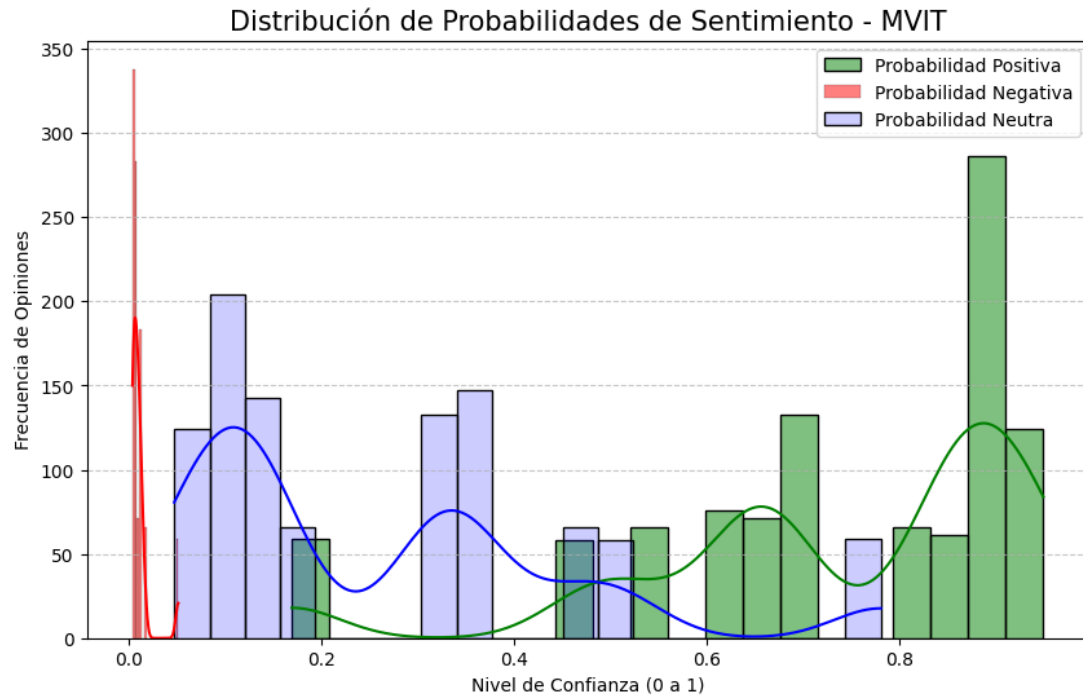
- Si una opinión tiene una valoración de 1 estrella pero el modelo le asigna un POS de 0.90, probablemente sea **sarcasmo** o un error del modelo.
- Casos Ambiguos: Si POS es 0.40 y NEU es 0.35, la predicción es "débil". Se puede utilizar para ignorar predicciones con baja confianza.

```
def obtener_probas(texto):  
    res = analyzer.predict(texto)  
    return res.probas  
  
# Aplicar al DF (tarda un poco en analizar  
todas las opiniones)  
df_probasmv =  
df_datamv["comentario"].apply(obtener_proba  
s).apply(pd.Series)  
  
df_probaspu =  
df_datapu["comentario"].apply(obtener_proba  
s).apply(pd.Series)
```

Añadir las columnas generadas al DataFrame de datos.

Dibujar en un histograma la distribución de probabilidades de sentimiento.

- Picos en los extremos (0 y 1): el modelo está muy seguro de sus predicciones (muy positivo o muy negativo).
- Picos en el centro (0.4 - 0.6): Indica ambigüedad, el modelo no sabe distinguir bien el sentimiento en esos textos (suelen ser opiniones neutras o confusas).
- Solapamiento: si ambas curvas se cruzan mucho en valores altos, puede tratarse de textos con sentimientos encontrados (mixtos).



Minería de textos – Análisis de sentimientos

Emociones semánticas

RoBERTuito



Emociones detectadas

- Anger: ira, enfado
- Disgust: asco, desagrado, frustración
- Fear: miedo
- Joy: alegría, felicidad
- Sadness: tristeza
- Surprise: sorpresa
- Neutral

Crear nuestro propio detector

```
EMOCIONES = {  
    "entusiasmo": ["motivado", "ganas", "encanta", "sorprendió", "muy satisfecho"],  
    "confianza": ["seguro", "estable", "fiable", "con confianza"],  
    "frustración": ["frustró", "estrés", "fallaba", "decepcionante"],  
    "decepción": ["decepcionado", "esperaba más", "no cumplió"],  
    "indiferencia": ["ni me", "sin más", "no destaca", "correcto"],  
}
```

```
def detectar_emocion_semantica(texto):  
    texto = texto.lower()  
    for emocion, palabras in EMOCIONES.items():  
        if any(p in texto for p in palabras):  
            return emocion  
    return "neutral_emocional"  
  
df["emocion_semantica"] = df["comentario"].apply(detector_emocion_semantica)
```



```
# Inicializar el analizador de emociones en español
emotion_analyzer = create_analyzer(task="emotion", lang="es")
```

```
def analizar_emocion(texto):
    resultado = emotion_analyzer.predict(texto[:512])
    return resultado.output, resultado.probas
```

```
emocionespu = df_datapu["comentario"].apply(analizar_emocion)

df_datapu["emocion"] = emocionespu.apply(lambda x: x[0])
```

```
print(df_datapu["emocion"].value_counts())

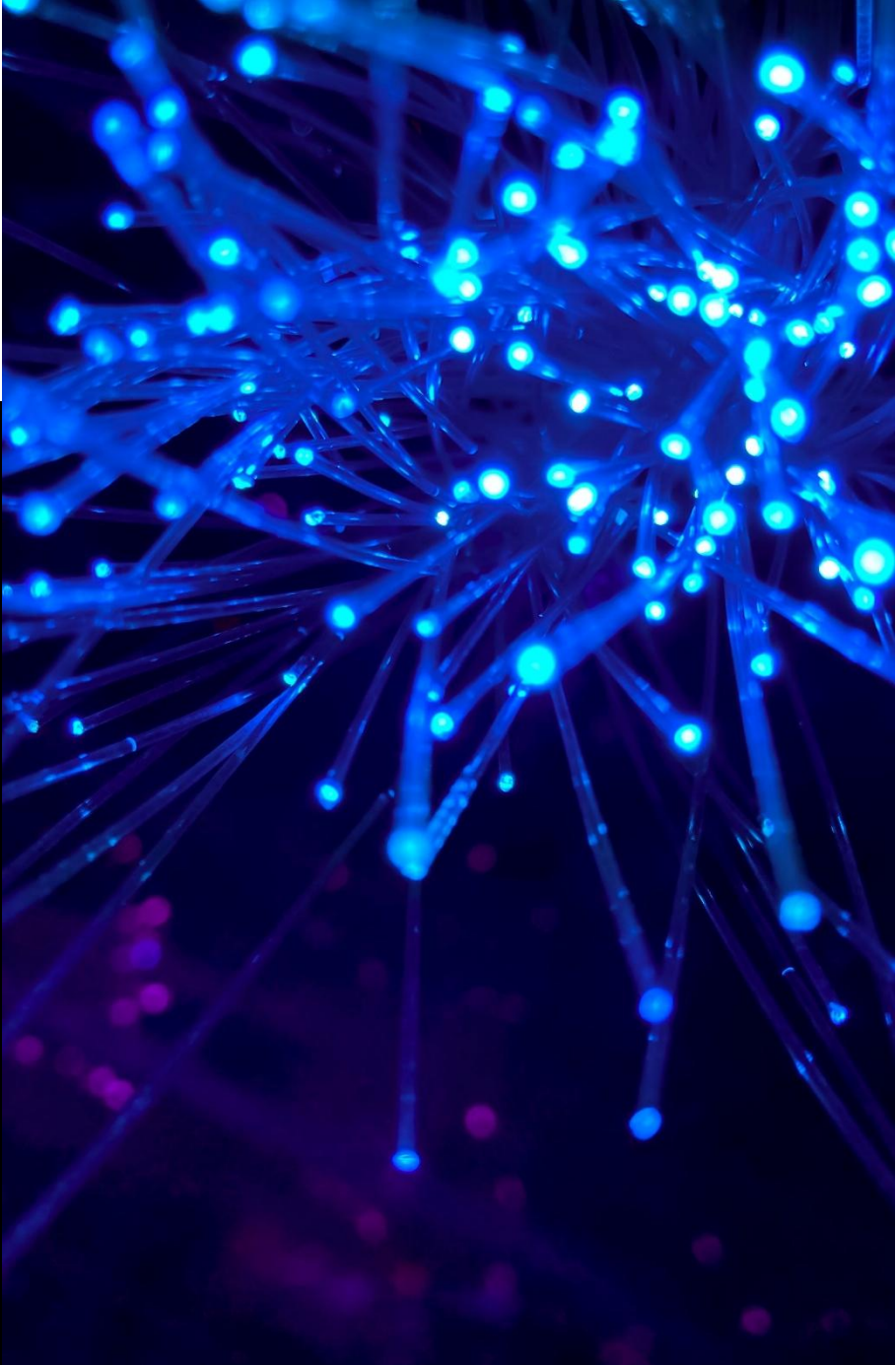
# En función de las puntuaciones
print(df_datapu.groupby("emocion")['valoracion'].mean())

print(pd.crosstab(df_datapu["valoracion"],df_datapu["emocion"]))
```

```
emocion
others      182
joy          77
sadness     33
surprise     8
Name: count, dtype: int64

emocion
joy          4.285714
others       3.318681
sadness      1.696970
surprise     4.375000
Name: valoracion, dtype: float64

emocion  joy  others  sadness  surprise
valoracion
1         0      5       10        0
2         0     32       23        0
3         0     70        0        0
4        55     50        0         5
5        22     25        0         3
```



MongoDB: Vector Search

Datos

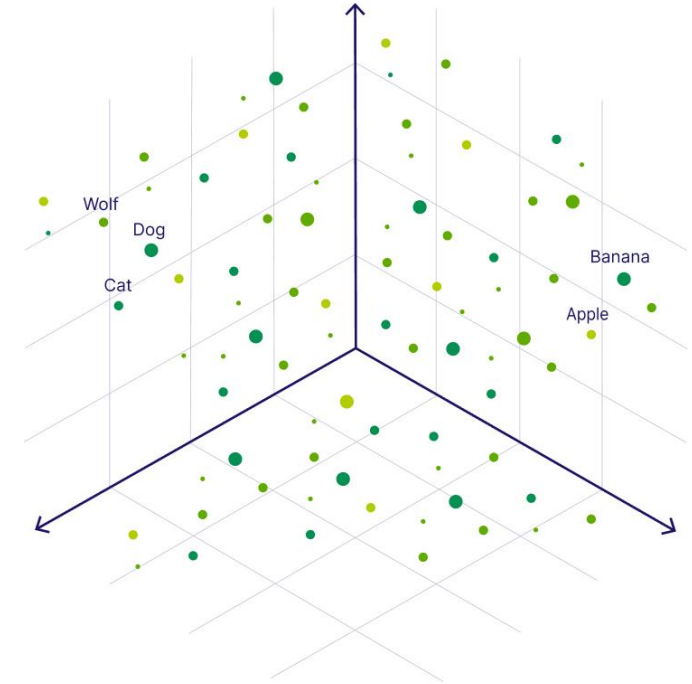
Limpiar & Estructurar

- palabras “sin contenido”
- signos de puntuación
- lematización

Añadir tags

Transformar mediante
algoritmos -> convertir en
vectores numéricos

i



Un vector (en el contexto de la inteligencia artificial) es simplemente una lista de números que representa el significado semántico de una imagen, una frase, audio e incluso vídeo.

BBDD vectoriales _ Búsquedas de similitud

Algoritmos matemáticos:

- Similitud del coseno: se basa en el ángulo entre dos vectores, sin importar su tamaño.
- Distancia euclidiana: distancia en línea recta entre dos vectores en el espacio.
- Similitud del producto de puntos: mide el producto de las magnitudes de los vectores y del coseno entre ellos.

Los datos se convierten a vectores utilizando modelos de codificación.

Similitud

Coseno

$$\text{score} = (1 + \cosine(v1,v2)) / 2$$

Euclidiana

$$\text{score} = 1 / (1 + euclidean(v1,v2))$$

Producto escalar

$$\text{score} = (1 + \text{dotProduct}(v1,v2)) / 2$$



Para aprender más sobre las matemáticas tras los algoritmos de puntuación.

https://lucene.apache.org/core/3_5_0/scoring.html

1. Conectar a MongoDB

2. Crear campos vectoriales en los documentos.

Debe usarse un modelo de incrustación vectorial (***Embedding Model***).

Vamos a usar para español el modelo [paraphrase-multilingual-mpnet-base-v2](#), con 768 dimensiones que tiene alta precisión.

Si no se tiene instalado sentence_transformers

`pip install -U sentence-transformers`

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
```

```
4 model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
5
✓ 26.2s
```

modules.json: 100%  229/229 [00:00<00:00, 28.6kB/s]

C:\Users\carne\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\Python312\python.exe: warning: Setuptools is not installed. This may cause problems for Python packages. To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. warnings.warn(message)

config_sentence_transformers.json: 100%  122/122 [00:00<00:00, 3.90kB/s]

README.md:  5.12k/? [00:00<00:00, 392kB/s]

sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 3.37kB/s]

config.json: 100%  723/723 [00:00<00:00, 63.3kB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to local storage.

model.safetensors: 100%  1.11G/1.11G [00:20<00:00, 52.6MB/s]

tokenizer_config.json: 100%  402/402 [00:00<00:00, 43.9kB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to local storage.

sentencepiece.bpe.model: 100%  5.07M/5.07M [00:00<00:00, 66.5MB/s]

tokenizer.json:  9.08M/? [00:00<00:00, 35.9MB/s]

special_tokens_map.json: 100%  239/239 [00:00<00:00, 28.8kB/s]

config.json: 100%  190/190 [00:00<00:00, 29.2kB/s]

3. Vectorizar y actualizar documentos

```
for doc in col_opispu.find({"comvector": {"$exists": False}}):
    vector = model.encode(doc['comentario']).tolist()
    col_opispu.update_one(
        {'_id': doc['_id']},
        {'$set': {'comvector': vector}}
    )
```

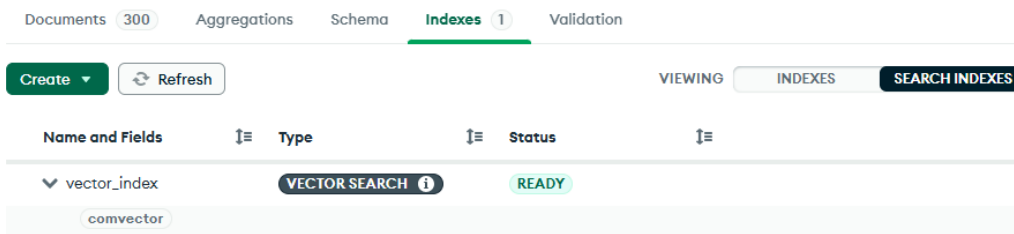
Comprobamos que se ha incluido
el campo correctamente en los
documentos de la colección.

```
_id: ObjectId('69e1ece43f996ed79a3dcd51')
id: 8
producto: "Pulsómetro X5"
valoracion: 4
comentario: "Desde que lo uso entreno con m
fecha: "2025-06-08"
recomendaria: true
ficticia: true
tokens: Array (8)
comvector: Array (768)
0: -0.046623557806015015
1: 0.12193504720926285
2: -0.010962248779833317
3: -0.06287243962287903
4: 0.025853971019387245
5: -0.010727115906774998
6: -0.03869613632559776
7: -0.00298150721937418
8: -0.07066172361373901
9: 0.1020912155508995
10: 0.041318535804748535
11: 0.07513558119535446
12: -0.08120444416999817
13: 0.12626171112060547
14: 0.016217747703194618
15: -0.04601012170314789
16: 0.03983800485730171
17: 0.01500000000000000
```


4. Crear un índice para la búsqueda vectorial en MongoDB

Es necesario indicar el nombre del campo, el tipo de similaridad y el número de dimensiones del vector.

Una vez finalizado, podemos ver el índice creado en Compass, en *search indexes*.



```
from pymongo.operations import SearchIndexModel
import time

# Create your index model, then create the search index
search_index_model = SearchIndexModel(
    definition = {
        "fields": [
            {
                "type": "vector",
                "path": "comvector",
                "similarity": "cosine",
                "numDimensions": 768
            }
        ]
    },
    name="vector_index",
    type="vectorSearch"
)
result = col_opispu.create_search_index(model=search_index_model)

# Wait for initial sync to complete
print("Polling to check if the index is ready. This may take up to a minute.")
predicate=None
if predicate is None:
    predicate = lambda index: index.get("queryable") is True

while True:
    indices = list(col_opispu.list_search_indexes(result))
    if len(indices) and predicate(indices[0]):
        break
    time.sleep(5)
print(result + " is ready for querying.")
```

5. Ejecutar la búsqueda: entender los parámetros

Se hace con el pipeline de agregación, con \$vectorSearch

```
{  
  "$vectorSearch": {  
    "exact": true | false,  
    "filter": {<filter-specification>},  
    "index": "<index-name>",  
    "limit": <number-of-results>,  
    "numCandidates": <number-of-candidates>,  
    "path": "<field-to-search>",  
    "queryVector": [<array-of-numbers>]  
  }  
}
```

```
{
  "$vectorSearch": {
    "exact": true | false,
    "filter": {<filter-specification>},
    "index": "<index-name>",
    "limit": <number-of-results>,
    "numCandidates": <number-of-candidates>,
    "path": "<field-to-search>",
    "queryVector": [<array-of-numbers>]
  }
}
```

- False: búsqueda ANN (Approximate Nearest Neighbor). Valor por defecto.
- True: búsqueda ENN (Exact Nearest Neighbor)

* Campo requerido si se omite el nº de candidatos

Búsqueda **ANN** (Approximate Nearest Neighbor)

Es una aproximación más eficiente que prioriza obtener resultados más rápidos, en lugar de exactos.

Se basa en búsquedas con métodos tipo grafo, en las que los puntos son nodos de un grafo y la búsqueda de vecinos un problema de búsqueda de caminos entre grafos.

Los algoritmos de búsqueda ANN se diseñan para minimizar en estos casos los problemas derivados de los mínimos locales, es decir, para incluir en la consideración los puntos inmediatos o muy cercanos.

Image recognition

In computer vision use cases, ANN can rapidly identify images with similar features from a vast dataset. This capability is invaluable in applications such as object recognition or categorizing images based on content similarity.

Music streaming services

For instance, in a music streaming service, ANN can be used to recommend songs that align with a user's preferences, even if they are not an exact match.

Medical imaging

In healthcare, ANN assists in quickly identifying diagnostic images similar to a query, improving the speed and accuracy of patient diagnoses.

Búsqueda **ENN** (Exact Nearest Neighbor)

Es una aproximación más enfocada en obtener resultados exactos, y más costosa en recursos de computación.

Se suele combinar con número límite de resultados. ANN con n° de candidatos.

Los algoritmos de búsqueda ENN devolverán los resultados más cercanos.

Small-scale vector data

For datasets under 10,000 vectors, the linear time complexity of ENN vector search makes it a viable option, especially considering the added development complexity of tuning ANN parameters.

Recall benchmarking of ANN queries ("comparativas")

This will allow you to have much greater confidence that your ANN queries are accurate since you can build quantitative benchmarks as your data evolves.

Multi-tenant architectures ("arqu. Multiusuario")

In cases where the overall vector collection is large (in the millions) but the number of vectors per tenant is small (a few thousand), ANN's accuracy suffers when applying highly selective filters. ENN vector search thrives in this multi-tenant scenario, delivering precise results even with small result sets.

```
{  
  "$vectorSearch": {  
    "exact": true | false,  
    "filter": {<filter-specification>},  
    "index": "<index-name>",  
    "limit": <number-of-results>,  
    "numCandidates": <number-of-candidates>,  
    "path": "<field-to-search>",  
    "queryVector": [<array-of-numbers>]  
  }  
}
```

Número de vecinos cercanos a usar durante la búsqueda.

- ≤ 10.000
- > limit

Debe ajustarse correctamente para no provocar una mayor latencia.

* Campo requerido si se omite exact


```
{  
  "$vectorSearch": {  
    "exact": true | false,  
    "filter": {<filter-specification>},  
    "index": "<index-name>",  
    "limit": <number-of-results>,  
    "numCandidates": <number-of-candidates>,  
    "path": "<field-to-search>",  
    "queryVector": [<array-of-numbers>]  
  }  
}
```

Filter: prefiltrado de datos en MQL
(MongoDB query language).

Index: *vector search index* a utilizar.

Limit (int): nº máx de resultados.

Path: tipo de campo vectorial indexado en
el que buscar.

QueryVector: elemento de búsqueda en
formato de array de números o BSON
binario.

5. Ejecutar la búsqueda: Método para buscar

Score: valor entre 0 y 1

- Depende de la posición de los elementos y del índice creado.
- Este algoritmo normaliza la puntuación considerando el tipo de similitud entre el vector de documento (v1) y el vector de consulta (v2). Se ajusta para asegurar un valor entre 0 y 1.

```
def buscar_opinion(query_texto, limit = 3):  
    # Vectorizar la pregunta del usuario  
    query_vector = model.encode(query_texto).tolist()  
  
    # Ejecutar búsqueda vectorial  
    pipeline = [  
        {  
            "$vectorSearch": {  
                "exact": False,  
                "index": "vector_index",  
                "path": "comvector",  
                "queryVector": query_vector,  
                "numCandidates": 100, # Candidatos a evaluar  
                "limit": limit # Resultados finales  
            }  
        },  
        {  
            "$project": {  
                "id": 1,  
                "comentario": 1,  
                "valoracion": 1,  
                "score": {"$meta": "vectorSearchScore"} #  
                Ver qué tan similar es  
            }  
        }  
    ]  
  
    return list(col_opispu.aggregate(pipeline))
```

5. Ejecutar la búsqueda: ejemplos

```
resultados = buscar_opinion(texto_busc, 20)
unq_com = []
for res in resultados:
    if res['comentario'] not in unq_com:
        unq_com.append(res['comentario'])
        print(f"Opinión: {res['id']} - {res['comentario']} Valoración {res['valoracion']} (Score: {res['score']})")
```

```
texto_busc = "buen funcionamiento"
```

```
Opinión: 40 - Después de varias semanas de uso intensivo sigue funcionando como el primer día. Muy buen producto. Valoración 5 (Score: 0.8046408891677856)
Opinión: 205 - Funciona correctamente, aunque a veces siento que le falta algo para destacar realmente. Valoración 3 (Score: 0.792067289352417)
Opinión: 211 - Para un uso ocasional está bien, aunque creo que hay margen de mejora en la aplicación. Valoración 2 (Score: 0.7764064073562622)
```

```
texto_busc = "dispositivo fallido"
```

```
Opinión: 286 - Me arrepiento de la compra. Para mi uso no resultó nada práctico. Valoración 2 (Score: 0.7167208194732666)
Opinión: 246 - Después de unas semanas dejó de sincronizar correctamente y fue muy frustrante. Valoración 2 (Score: 0.7043067812919617)
Opinión: 271 - Acabé desmotivándome porque perdía la conexión justo cuando más lo necesitaba. Valoración 2 (Score: 0.7021485567092896)
Opinión: 287 - Intenté darle varias oportunidades, pero siempre acababa fallando en momentos clave. Valoración 2 (Score: 0.68819260597229)
```

Los umbrales de similaridad pueden variar dependiendo del modelo de incrustación vectorial utilizado.

paraphrase-multilingual-mpnet-base-v2:

- Umbral de alta precisión: > 0.85
- Rango óptimo: $0.7 - 0.85$
- Más o menos relacionado: $0.6 - 0.7$
- Ruido < 0.6

En cualquier caso, los valores experimentales al probar con diferentes búsquedas van a determinar el resultado.

Comentarios y preguntas